

Unit-II: Server-Side Programming with Python (8 hours)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Control Structures

- Control structures in Python manage the flow of execution within a program.
- They enable decisions, repetitions, and alterations in the sequence of code execution.
- The primary control structures in Python are conditional statements and loops.
- Conditional statements allow different blocks of code to be executed based on whether a condition is true or false.

Control Structures

- **if statement:** Executes a block of code if a condition is true.

For example,

```
x = 10
```

```
if x > 5:
```

```
    print("x is greater than 5")
```

Control Structures

- **elif statement:** Checks an additional condition if the preceding if condition is false.

For example,

```
x = 3
```

```
if x > 5:
```

```
    print("x is greater than 5")
```

```
elif x > 2:
```

```
    print("x is greater than 2 but not greater than 5")
```

Control Structures

- **else statement:** Executes a block of code if all preceding if and elif conditions are false.

For example,

```
x = 1
```

```
if x > 5:
```

```
    print("x is greater than 5")
```

```
elif x > 2:
```

```
    print("x is greater than 2 but not greater than 5")
```

```
else:
```

```
    print("x is not greater than 2")
```

Loops

- Loops allow a block of code to be executed repeatedly.
- **for loop:** Iterates over a sequence (like a list, tuple, or string) or other iterable objects.

For example,

```
for i in range(5):  
    print(i)
```

Loops

You can use the `range()` function with a step argument to achieve this. The `range()` function takes three arguments: start, stop, and step. The step argument specifies the increment value.

```
for i in range(0, 10, 2):  
    print(i)
```

Loops

- **while loop:** Executes a block of code as long as a condition is true.

- For example,

```
x = 0
```

```
while x < 5:
```

```
    print(x)
```

```
    x += 1
```


Loop Control Statements

- Loop control statements alter the execution of loops.
- **break statement:** Terminates the loop prematurely.

For example,

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

Loop Control Statements

- **continue statement:** Skips the rest of the current iteration and proceeds with the next iteration of the loop.
- For example,
 for i in range(10):
 if i % 2 == 0:
 continue
 print(i)

Loop Control Statements

- **pass statement:** It is used as a placeholder for future code.
- When the pass statement is executed, nothing happens, but you avoid getting an error when empty code is not allowed.

- For example,

```
for i in range(5):
```

```
    if i == 2:
```

```
        pass # Do nothing for i == 2
```

```
    else:
```

```
        print(i)
```

List Comprehension

- List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Example:

- Based on a list of fruits, you want a new list, containing only the fruits with the letter "a" in the name.
- Without list comprehension you will have to write a for statement with a conditional test inside:

List Comprehension

for example,

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = []  
for x in fruits:  
    if "a" in x:  
        newlist.append(x)  
print(newlist)
```

Output:

```
['apple', 'banana', 'mango']
```

List Comprehension

- With list comprehension you can do all that with only one line of code:
- For example,

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = [x for x in fruits if "a" in x]  
print(newlist)
```

Output:

```
['apple', 'banana', 'mango']
```

List Comprehension

The Syntax

`newlist = [expression for item in iterable if condition == True]`

- The return value is a new list, leaving the old list unchanged.

Condition

- The condition is like a filter that only accepts the items that evaluate to True.

Example

- Only accept items that are not "apple":

`newlist = [x for x in fruits if x != "apple"]`

List Comprehension

- The condition if `x != "apple"` will return `True` for all elements other than "apple", making the new list contain all fruits except "apple".
- The condition is optional and can be omitted:

Example

- With no if statement:

```
newlist = [x for x in fruits]
```

Iterable

- The iterable can be any iterable object, like a list, tuple, set etc.

List Comprehension

Example

- You can use the range() function to create an iterable:

```
newlist = [x for x in range(10)]
```

```
# Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- Same example, but with a condition:

Example

- Accept only numbers lower than 5:

```
newlist = [x for x in range(10) if x < 5] # [0, 1, 2, 3, 4]
```

List Comprehension

Expression

- The expression is the current item in the iteration, but it is also the outcome, which you can manipulate before it ends up like a list item in the new list:

Example

- Set the values in the new list to upper case:
`newlist = [x.upper() for x in fruits]`

List Comprehension

- You can set the outcome to whatever you like:

Example

- Set all values in the new list to 'hello':

```
newlist = ['hello' for x in fruits]
```

- The expression can also contain conditions, not like a filter, but as a way to manipulate the outcome:

List Comprehension

Example

- Return "orange" instead of "banana":

```
newlist = [x if x != "banana" else "orange" for x in fruits]
```

- The expression in the example above says:
- "Return the item if it is not banana, if it is banana return orange".